

MEDIATEK

MT8668 Hypervisor GNSS User Manual

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	Ashin Wang (王薇)	正式版

目录

版本记录	2
目录	3
图片目录	4
表格目录	4
1 GNSS	5
1.1 概述	5
1.1.1 简单介绍	5
1.1.2 GNSS 名词解释	5
1.2 架构/进程概述	5
1.2.1 GNSS 架构	5
1.3 配置/客制化指南	6
1.3.1 固定速率配置	6
1.3.2 多卫星导航系统配置	7
1.4 GNSS API	7
1.4.1 Code Path	7
1.4.2 GpsInterface	7
1.4.3 CallBack	8
1.4.4 Demo Code	9
1.5 常见问题/故障排除	9
1.5.1 Log 相关问题	9
1.5.2 测试相关问题	10
1.5.3 GNSS Path	11
附件一 附加条款	12

图片目录

图 1-1. GNSS 架构.....6

图 1-2. 固定速率配置.....6

图 1-3. GNSS 配置.....7

图 1-4. MnlD Test.....11

表格目录

表 1-1. 名词解释.....5

1 GNSS

1.1 概述

1.1.1 简单介绍

本章节介绍 MT8668 GNSS 的基本功能以及常见问题的解决方法。

1.1.2 GNSS 名词解释

表 1-1. 名词解释

缩略词	全称	释义
COLD start	–	有时间辅助资讯，终端用户不会遇到该场景。
FULL start	–	没有任何的辅助资讯，相当于终端用户第一次买到手机后使用定位应用的场景。
GNSS	Global Navigation Satellite System	全球导航卫星系统
Hot start	–	有所有的辅助资讯，终端用户此次定位距离上次定位小于 2 ~ 4 小时。
NMEA	National Marine Electronics Association	用于在海洋电子设备之间进行数据交换的通信协议，广泛应用于 GPS/GNSS 接收器数据输出。
TTFF	Time To First Fix	导航设备从开机到成功获取第一次有效定位数据所需要的时间。
WARM start	–	有时间和位置辅助资讯，终端用户此次定位距离上次定位超过 2 ~ 4 个小时。

1.2 架构/进程概述

1.2.1 GNSS 架构

MT8668 GNSS architecture 如图所示：

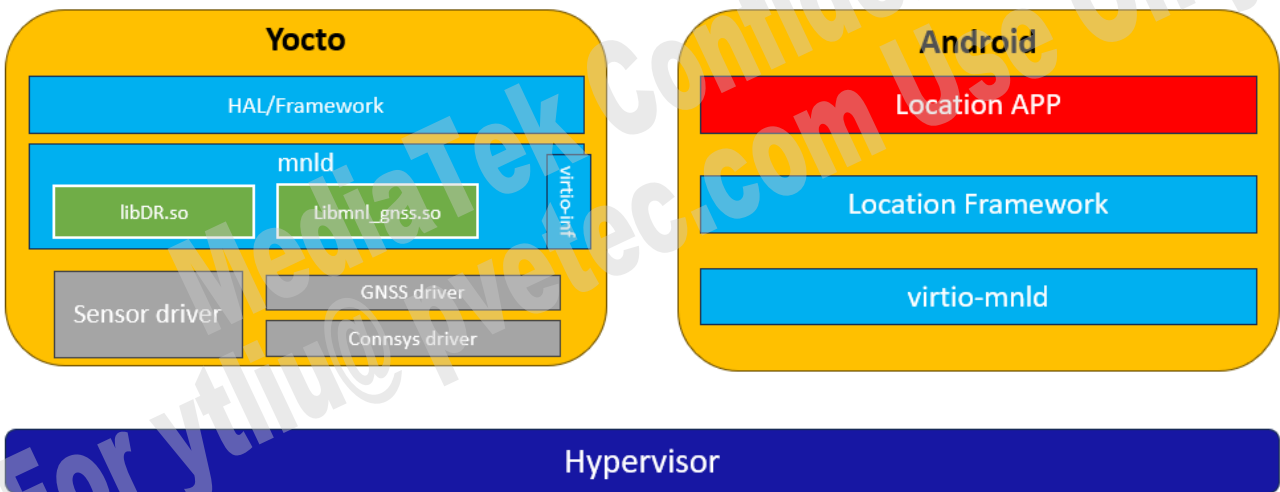


图 1-1. GNSS 架构

1.3 配置/客制化指南

1.3.1 固定速率配置

固定速率（Fix Rate）是指 GNSS 上报位置信息的速率，目前 MT8668 可以支持的固定速率包括：1Hz、2Hz、5Hz 和 10Hz，默认配置是 1Hz 输出。修改固定速率的方法如下：

方法 1：修改代码，配置 fix_interval 参数。fix_interval = 100 对应 10Hz；fix_interval = 1000 对应 1Hz。

需要改在 Yocto 端/ src/connectivity/gps/4.0/mtk_mnld/mnld_entity/src/gps_controller.c

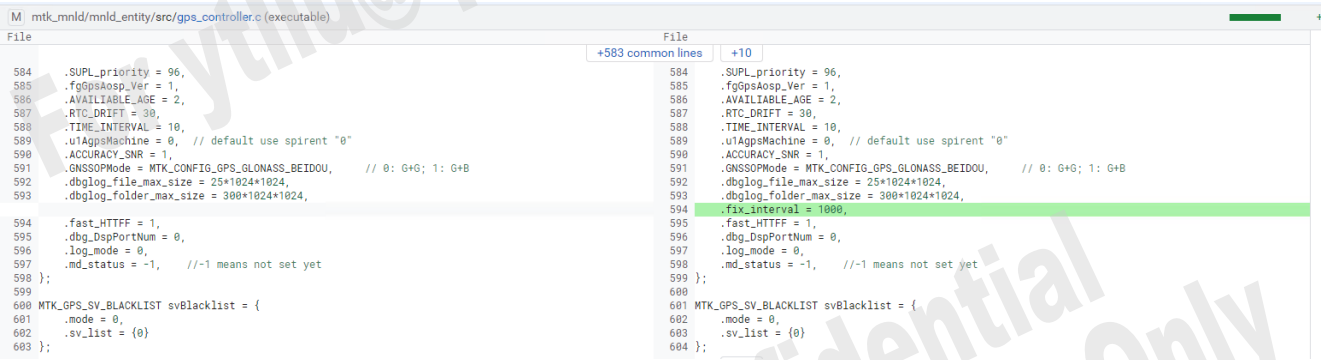


图 1-2. 固定速率配置

方法 2：动态修改配置文件，重启 GNSS 后生效，在 Yocto 端修改。

命令：

```
echo fix_interval=1000 >> /data/etc/gnss/mnl.prop /
```

配置成 1Hz，重启 GPS 后生效。

1.3.2 多卫星导航系统配置

MT8668 支持 GPS + GLONASS + Galileo + BeiDou 多卫星导航定位系统，gnssopmode 默认配置成

MTK_CONFIG_GPS_GLONASS_BEIDOU_GALILEO_NAVIC（默认配置的 GNSS 性能最佳，建议使用默认配置）。

```

885 typedef enum
886 {
887     MTK_CONFIG_GPS_GLONASS = 0,
888     MTK_CONFIG_GPS_BEIDOU,
889     MTK_CONFIG_GPS_GLONASS_BEIDOU,
890     MTK_CONFIG_GPS_ONLY,
891     MTK_CONFIG_BEIDOU_ONLY,
892     MTK_CONFIG_GLONASS_ONLY,
893     MTK_CONFIG_GPS_GLONASS_BEIDOU_GALILEO,
894     MTK_CONFIG_GPS_GALILEO,
895     MTK_CONFIG_GPS_GLONASS_GALILEO,
896     MTK_CONFIG_GALILEO_ONLY,
897     MTK_CONFIG_GPS_GLONASS_BEIDOU_GALILEO_NAVIC,
898     MTK_CONFIG_GNSS_END
899 } MTK_GNSS_CONFIGURATION;

```

图1-3. GNSS 配置

1.4 GNSS API

1.4.1 Code Path

Code path:

/src/connectivity/gps/4.0/gps_hal

1.4.2 GpsInterface

```

typedef struct {
    size_t      size;      //设置 sizeof(GpsInterface)

    int  (*init)( GpsCallbacks_ext* callbacks );      //初始化 callback
    int  (*start)( void );      //gps start
    int  (*stop)( void );      //gps stop
    void (*cleanup)( void );      //cleanup gps

    int  (*inject_time)(GpsUtcTime time, int64_t timeReference,int uncertainty);
    //注入 utc time

    int  (*inject_location)(double latitude, double longitude, float accuracy);
    //将当前位置从另一个位置提供者注入， Latitude and longitude 以度为单位，精度以米为单位

    void (*delete_aiding_data)(GpsAidingData flags);
    //指定下一次调用启动时不会使用在 flags 中定义的信息。对于冷启动，会传递 GPS_DELETE_ALL。

```

```

int    (*set_position_mode) (GpsPositionMode mode, GpsPositionRecurrence recurrence,
                             uint32_t min_interval, uint32_t preferred_accuracy, uint32_t preferred_time,
                             bool lowPowerMode);
    //主要是设置 nmea 和 location 的上报频率: min_interval

const void* (*get_extension) (const char* name);
    //get a pointer to extension information

int    (*inject_fused_location) (double latitude, double longitude, float accuracy);
    //注入 fused location

int    (*fix_interval) (uint32_t interval);
    //动态设置上报频率, 100:10hz, 1000:1hz

int    (*op_mode) (uint8_t opmode);
    //动态设置 opmode

int    (*elevation_angle) (uint8_t angle);
    //动态设置 elevation angle constraint

int    (*l5_disable) (bool disable);
    //动态设置 L5 enable 或 disable

int    (*gps_start_week_num) (uint32_t week);
    //动态设置 gps start week num

```

1.4.3 Callback

```

typedef struct {
    gps_location_ext_callback location_cb;    //上报 location 的 callback, 从这里获取 location, 比较常用
    gps_status_callback status_cb;           //gps status callback, gps 状态如 start、stop
    gps_sv_status_callback sv_status_cb;     //sv status callback
    gps_nmea_callback nmea_cb;              //上报 nmea 的 callback, 从这里获取 nmea, 比较常用
    gps_set_capabilities set_capabilities_cb; //capabilities 在 xts 认证会用到, 获取支持的 capabilities 再去跑特定测项

    gps_acquire_wakelock acquire_wakelock_cb; //请求 wakelock 的 callback
    gps_release_wakelock release_wakelock_cb; //是否 wakelock 的 callback
    gps_create_thread create_thread_cb;       //create thread 的 callback
    gps_request_utc_time request_utc_time_cb; //获取 utc time 的 callback
    gnss_set_system_info set_system_info_cb;  //set system info 的 callback
    gnss_sv_status_ext_callback gnss_sv_status_cb; //获取 gnss sv_status_ext 的 callback

    gnss_set_name_callback set_name_cb;       //set nmea callback

```

```

gnss_request_location_callback request_location_cb;    //request location callback
gps_location_ext_callback agps_location_cb;           //获取 agps location

gnss_output_rtc3_callback output_rtc3_cb;            //获取 rtc3 的 callback
gnss_set_signal_type_capabilities_callback set_signal_type_callback;    //set signal type
capabilities, 如 carrierfrequency

```

1.4.4 Demo Code

可以参考 `mnld_test` 的用法来调用 `GpsInterface` 和 `callback`

`Mnld_test` 路径: `src/connectivity/gps/4.0/gnss_test`

例如: 调用 `GpsInterface` 中的 `init` 和 `fix_interval`:

```

GpsInterface_ext* mnld_test_gpsinfs = NULL;
GpsCallbacks_ext* mnld_test_cbs = NULL;
mnld_test_cbs = (GpsCallbacks_ext*)calloc(1, sizeof(GpsCallbacks_ext));
mnld_test_cbs->size = sizeof(GpsCallbacks_ext);
struct gps_device_t_ext *gpsdev = NULL;

gpsdev = &linux_gps_device;    //定义变量初始化
mnld_test_gpsinfs = (GpsInterface_ext*)gpsdev->get_gps_interface(gpsdev);
if(mnld_test_gpsinfs != NULL)
{
    LOGI("mnld_test_gpsinfs is not null");
mnld_test_gpsinfs->init(mnld_test_cbs);
    mnld_test_gpsinfs->fix_interval(1000);
}

```

`Callback` 的使用也可以参考 `mnld_test`、`mnld_fm_gps_location_callback`、`mnld_fm_gps_status_callback`, 它们都是 `callback` 的接收函数, 例如:

```

GpsCallbacks_ext mnld_fm_gps_callbacks = {
    .size = sizeof(GpsCallbacks_ext),
    .location_cb = mnld_fm_gps_location_callback,
    .status_cb = mnld_fm_gps_status_callback,
    .....
}

```

建议细看 `mnld_test` 的 code, 有很具体的接口调用和 `callback` 使用。

1.5 常见问题/故障排除

1.5.1 Log 相关问题

- 联发科技工程师需要哪些 log 用于分析问题?

Android 端需要: /data/debuglogger/mobilelog

Yocto 端需要: /data/debuglogger/mobilelog, 以及 NMEA log

Yocto 端的 mobilelog 默认会打开, 开 NMEA log 需要创建 /data/etc/gnss/mnl.prop 文件并在里面写入:

```
debug.dbg2file=1
debug.filename=/data/debuglogger/gpsdebug.log
```

保存重启, NMEA log 就会出现在 /data/debuglogger/gpsdebug.log

- 如何连接 powergps

先输入如下命令

```
adb shell "echo \"pmtk.serial.port=7000\" > /data/etc/gnss/mnl.prop"
adb shell "echo \"debug_type=0\" >> /data/etc/gnss/mnl.prop"
adb shell "echo \"debug.debug_nmea=1\" >> /data/etc/gnss/mnl.prop"
adb shell "echo \"debug.dbg2file=1\" >> /data/etc/gnss/mnl.prop"
adb shell "echo \"debug.filename=/data/debuglogger/gpsdebug.log\" >> /data/etc/gnss/mnl.prop"
```

重启后再输入:

```
adb forward tcp:7000 tcp:7000
```

并使用 mnld_test 打开 GPS:

```
adb shell
mnld_test start c &
```

1.5.2 测试相关问题

- 测试前需要检查是否有卫星信号, 是否处于 open sky 的环境

测试 GNSS 搜星或定位功能, 信号需要 open sky 的环境下, 例如空旷的室外或者有信号放大器的实验室。

能够定位是有前提条件的 CNR 为 40~43dbm 的卫星要>6 颗。--->测 GNSS 一定要注意这个, 如不确认当前信号环境是否符合要求, 拿一个对比机放在同样的环境做对比。

- 如何测试 FULL start、WARM start、COLD start、HOT start 这几种启动方式的 TTFF?

Android 端请使用工程模式下的 YGPS 或使用如下 adb 命令打开 YGPS, 通过 FULL、COLD、WARM、HOT 按钮来测试。

```
adb shell am start com.mediatek.ygps/.YgpsActivity
```

Yocto 端可以使用 mnld_test

```
Start test(open gps): mnld_test -h
```

```
sh-3.2# mnld_test -h
mnld_test -h
mnld_test -h
MNLD client test :
-----
The command to start GNSS test:
mnld_test start [start type] [restart times] [restart interval]
[start type]:
h/H: hot start;
w/W: Warm start;
c/C: Cold start;
f/F: Full start
[restart times]: integer value, range is 0-1000, default is 0(no restart, always on).
[restart interval]: integer value, range is 0-3600, the unit is second; the default value is 60
-----
The command to stop GNSS test:
Ctrl+C or kill -9 `pidof mnld_test`
-----
The command to update network status:
mnld_test network [type] [roaming]
[type]: wifi, mobile, disable
[roaming]: roaming, the mobile network is in roaming state
sh-3.2#
```

图1-4. Mnld Test

根据提示，例如您需要测试冷启动，可以用如下命令：

```
mnld_test start c &
```

可以输入如下指令将 log 打在串口，可以看到 TTFF 等信息：

```
journalctl -f --no-tail -o short-precise |grep mnldtest
```

1.5.3 GNSS Path

Source code:

Android: vendor\mediatek\proprietary\hardware\connectivity\gnss

Yocto: src/connectivity/gps/4.0

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。